

StanzaOps – Hostel Operations & Revenue Intelligence Engine

Final Presentation Documentation & Technical Defence

Project Portfolio Presentation Document

1. Project Introduction

StanzaOps is a hostel operations and revenue intelligence project designed to organize resident, occupancy, payment, and operational information into a centralized analytics workflow. The project combines data cleaning, exploratory analysis, SQL-based analysis, and Power BI visualization to convert raw hostel records into meaningful business insights.

The project is designed for hostel management teams who need a clear view of resident volume, payment performance, deposits, occupancy, and rental information.

2. Business Problem

Hostel operations can involve large amounts of resident, room, payment, and identification information. When this information is maintained in inconsistent or manually managed formats, it becomes difficult to monitor payments, occupancy, and overall operational performance.

- Difficulty monitoring a large resident dataset efficiently.
- Need for consistent and validated resident and payment information.
- Difficulty identifying payment completion and partial-payment patterns.
- Need for quick KPI-based monitoring of revenue and occupancy.
- Need to convert raw operational records into management-friendly insights.

3. Project Objective

The primary objective is to build a reliable analytics workflow that cleans hostel data, validates important fields, performs SQL and statistical analysis, and presents the results through an interactive Power BI dashboard.

- Clean and standardize the hostel dataset.
- Validate identifiers, dates, categories, and important numeric fields.
- Analyze residents, payments, deposits, cities, states, and gender.
- Create business KPIs for management monitoring.
- Build a Power BI dashboard for clear decision support.
- Document the complete workflow for portfolio and technical review.

4. Proposed Solution

The solution follows a simple analytics pipeline:

1. Collect and inspect the hostel dataset.
2. Profile the data and identify quality issues.
3. Clean, standardize, and validate the dataset.
4. Perform exploratory and statistical analysis.
5. Load and analyze the data using SQL.
6. Create KPI measures and dashboard visuals in Power BI.
7. Document insights, limitations, and recommendations.
8. Publish the project artifacts through GitHub and the portfolio.

5. Dataset Overview

The cleaned StanzaOps dataset contains 10,000 resident records and 43 columns.

Attribute	Details
Rows	10,000
Columns	43
Numeric columns	15 (including integer and float fields)
Object/category/date-related columns	28
Primary resident identifier	Resident_ID
Important business fields	Monthly_Rent, Deposit_Amount, Amount_Paid, Balance_Amount
Operational fields	Room_Number, Bed_Number, Room_Type, Occupancy_Status, Bed_Status
Payment fields	Payment_Date, Payment_Method, Payment_Status

6. Data Cleaning & Validation

Data cleaning was performed to improve consistency, reliability, and suitability for SQL analysis and dashboard reporting.

- Converted relevant date columns such as Date_of_Birth, Check_In_Date, Payment_Date, and Joining_Date to datetime format.
- Checked Resident_ID for duplicates; no duplicate Resident_ID records were identified.
- Checked Mobile_Number for duplicates; no duplicate Mobile_Number records were identified.
- Standardized Gender values to Male and Female.
- Standardized Payment_Status categories to Paid and Partial.
- Kept Exit_Date as NULL where an exit date was not available because missing exit dates can represent active residents rather than bad data.
- Replaced blank Remarks values with 'No Remarks' where appropriate.
- Validated age values; the observed age range was 0 to 100 and no invalid age records were identified.
- Checked important categorical fields such as City, State, and Gender for consistency.

7. Cleaning Rules & Rationale

Field / Issue	Action	Reason
Resident_ID duplicates	Remove duplicate records	Protect unique resident-level analysis.
Mobile_Number duplicates	Check and remove duplicate records where applicable	Avoid repeated resident identities.
Date fields	Convert to datetime	Enable reliable date analysis.
Gender	Standardize categories	Prevent inconsistent grouping.
Payment_Status	Standardize Paid / Partial	Support payment KPI calculations.
Exit_Date missing	Keep NULL	Missing exit can represent an active resident.
Remarks missing	Use 'No Remarks'	Make blank operational notes explicit.

8. Exploratory & Statistical Analysis

Exploratory analysis focused on Age, Deposit_Amount, and Amount_Paid, along with segmentation by Gender, City, and State.

- Age was analyzed using summary statistics and distribution analysis.
- Deposit_Amount was examined for variation and outliers.
- Amount_Paid was compared across payment categories and geographic segments.
- Gender, City, and State were used to identify segment-level differences.
- Date fields were used to derive features such as Age, Birth_Year, Birth_Month, Birth_Day, Joining_Year, Joining_Month, and Joining_Day.

Outlier analysis used IQR and z-score approaches. No age outliers were detected by the applied methods.

Deposit_Amount showed no outliers because the dataset uses a constant deposit amount of ₹10,000. Amount_Paid showed limited variation that mainly reflects the payment categories.

9. Statistical Findings

- The dataset contains 10,000 residents.
- Male and female residents each account for 5,000 records.
- Deposit_Amount is constant at ₹10,000 per resident, producing a total deposit value of ₹100M.
- Amount_Paid totals ₹70M across the dataset.
- Payment status consists of 7,500 Paid records and 2,500 Partial records.
- City-level payment totals differ and can be investigated further using rent, room type, or other operational factors.
- State-level totals show Andhra Pradesh with 5,000 records, while Tamil Nadu and Telangana each have 2,500 records.

10. SQL Analysis

SQL was used to transform the cleaned dataset into business-oriented summaries and views. The SQL layer supports repeatable aggregation and provides structured outputs that can be consumed by reporting and dashboard workflows.

- Resident-level and payment-level aggregations were performed.
- Revenue was calculated from Amount_Paid.
- Deposit totals were calculated from Deposit_Amount.
- Payment status counts were used to support payment KPIs.
- Revenue was analyzed by gender, city, and state.
- SQL views were used to organize reusable reporting outputs such as payment summaries and geographic revenue summaries.

Example SQL concepts used in the project include GROUP BY, aggregate functions such as SUM and AVG, COUNT, CASE-based classification where required, and reusable SQL views.

11. Key SQL Results

Segment	Residents	Deposit	Amount Paid
Male	5,000	₹50M	₹40M
Female	5,000	₹50M	₹30M
Andhra Pradesh	5,000	₹50M	₹30M
Tamil Nadu	2,500	₹25M	₹20M
Telangana	2,500	₹25M	₹20M

12. KPI Definitions

KPI	Value	Definition
Total Residents	10,000	Total number of resident records.
Total Revenue	₹70M	Total Amount_Paid collected.
Total Deposit	₹100M	Total Deposit_Amount collected.
Payment Success Rate	75%	Share of residents with Paid payment status.
Occupancy Rate	100%	Occupancy percentage represented by the dataset.
Average Monthly Rent	₹8K	Average Monthly_Rent per resident.

13. Power BI Dashboard

The Power BI dashboard is the presentation layer of the project. It converts the analytical results into an easy-to-read management view.

- KPI cards display the main business metrics.
- Payment status visuals show the split between Paid and Partial residents.
- Geographic analysis supports comparison across cities and states.
- Resident segmentation supports gender-based analysis.
- Revenue and payment measures provide a financial view.
- The dashboard is designed to support fast management-level interpretation rather than requiring users to read raw tables.

14. Business Insights

- The project contains 10,000 resident records, providing a substantial operational dataset for analysis.
- Total amount collected is ₹70M against ₹100M in recorded deposits.
- The payment success rate is 75%, meaning 25% of residents are represented by Partial payment status.
- The dataset indicates 100% occupancy, suggesting full utilization under the defined occupancy measure.
- Male and female resident counts are balanced at 5,000 each.
- Andhra Pradesh contains 5,000 records, making it the largest state segment in the dataset.
- City-level differences in Amount_Paid suggest that further analysis of room type, rent, and resident segment could reveal drivers of payment variation.

15. Recommendations

9. Monitor the 25% Partial-payment segment and create a focused payment follow-up process.
10. Track payment performance by city, room type, and rent level to identify operational patterns.
11. Maintain standardized data-entry rules for resident, payment, and occupancy fields.
12. Continue monitoring occupancy and bed availability so that full occupancy does not hide capacity constraints.
13. Introduce periodic dashboard reviews for management to monitor revenue, payment completion, and occupancy.
14. Expand future analysis with historical monthly data to identify trends instead of relying only on a single dataset snapshot.

16. Dataset Limitations

- The dataset represents a specific analytical snapshot and may not capture long-term time trends.
- Deposit_Amount is constant, so it provides limited analytical variation.
- Exit_Date is missing for all records in the observed dataset, limiting historical exit/turnover analysis.
- The dataset may not contain enough historical transaction detail to analyze monthly revenue trends.
- A 100% occupancy result should be interpreted according to the project's occupancy definition and available capacity fields.
- Further validation with real operational source systems would be required before production deployment.

17. Final Conclusion

StanzaOps demonstrates an end-to-end data analytics workflow for hostel operations. The project connects data quality, statistical analysis, SQL reporting, KPI design, and Power BI visualization into one portfolio-ready solution. The final deliverable is intended to demonstrate both technical capability and the ability to translate data into business-oriented conclusions.